

0.72. Plus vite sur la grille : `deplace_croix` à 100

Le 0.71 avec un seul nombre changé : la vitesse passe de 250 à 100 millisecondes. La lettre fait 10 cases par seconde au lieu de 4.

1. Ce qu'il faut comprendre

C'est le 0.71, avec un seul changement : la vitesse, 4e réglage de `deplace_croix`, passe de 250 à 100.

Ce qui est nouveau ici : choisir sa vitesse. 100 millisecondes entre deux pas : **10 cases par seconde**, deux fois et demie plus vite qu'au 0.71. 500 ferait 2 cases par seconde, très lentement.

Plus le nombre est petit, plus ça va vite, puisque c'est le temps d'attente entre deux pas.

Trop vite, c'est difficile à diriger : à 50 (20 cases par seconde), la lettre traverse l'écran en une seconde, et il devient dur de s'arrêter sur la bonne case. Essaie, et choisis ce qui convient à ton jeu.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1181 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	73 o	← la console, à l'allumage – la mise en route de la console
<code>deplace_croix()</code>	217 o	← <code>main()</code> – une fonction fournie par le langage
<code>main()</code>	32 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> – attend de pouvoir écrire une case
AdresseCase	10 o	← <code>effacer()</code> , <code>poser()</code> – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← <code>effacer()</code> – efface des cases
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← <code>bouton()</code> – lit les huit boutons
Tuiles	704 o	← <code>CopierTuiles</code> – la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>bouton()</code>	× 4	20 o sur place, dans <code>deplace_croix()</code> + <code>LireManette</code> (37 o)
<code>effacer()</code>	× 1	15 o sur place, dans <code>deplace_croix()</code> + <code>AdresseCase</code> (10 o), <code>EffacerCases</code> (12 o)
<code>poser()</code>	× 1	13 o sur place, dans <code>deplace_croix()</code> + <code>AdresseCase</code> (10 o)
<code>image()</code>	× 1	3 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)
<code>images()</code>	× 1	2 o sur place, dans <code>deplace_croix()</code>

```
Tes fonctions appelées :  
deplace_croix() x 1    19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix>    // une tuile qui suit la croix, case par case  
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

```
uint8_t x = 9;            // la COLONNE de la lettre
```

3. UNE VARIABLE — Y

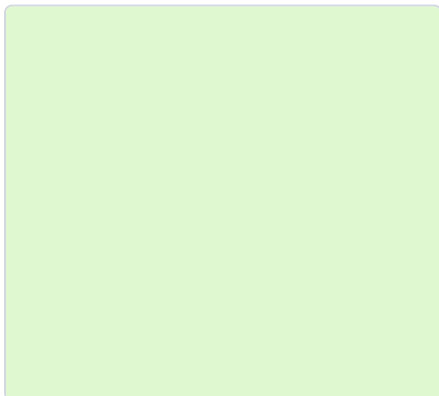
```
uint8_t y = 8;            // la LIGNE de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

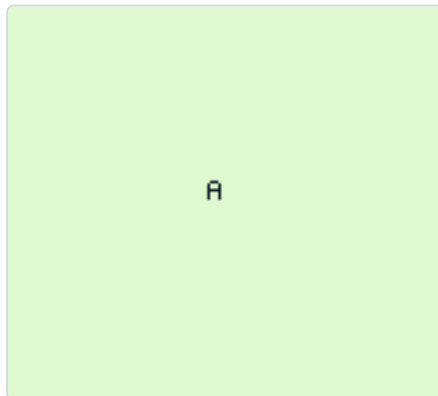
```
int main() {  
    while (true) {  
        image();  
  
        // Le changement : la vitesse, 250 → 100.  
        //  
        //  deplace_croix(x, y, ALPHABET[0], 100);  
        //  
        //  
        //      +-- vitesse : le temps entre deux pas.  
        //      Plus PETIT = plus VITE :  
        //      500 → 2 cases par seconde  
        //      250 → 4 cases par seconde (le 0.71)  
        //      100 → 10 cases par seconde (ici)  
        //      50  → 20 cases par seconde  
        deplace_croix(x, y, ALPHABET[0], 100);  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le même A qu'au 0.71, mais qui file sur la grille : 10 cases par seconde.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 30 images à droite : 5 pas, un toutes les 6 images ($x = 14$)

✓ le A est à sa nouvelle place

Ensuite : **0.72.1. Plus vite sur la grille : deplace_croix à 100 — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++